

小芽：宠物对战游戏里的 LLM 教练

设计与实现说明

周宇宸 腾讯 IEG 面试题提交稿 2026 年 9 月

本报告由 AI 辅助撰写。

1 题目与取舍

这道题要求给一个宠物对战游戏做一个 AI 教练，要有军师、陪练、老师三种角色感，参考方向列了 Agentic RL、RAG、ReAct、Memory Stream / Reflection，产品参考是和平精英的「小田」和王者荣耀的「灵宝」。我的理解是：要交付的是教练本身，游戏只是让教练有局面可看的那个环境；三种角色感也不是三种说话语气，而是三套不同的正确性标准——军师要算得准，还要说得有依据；老师只允许用当时看得见的信息讲话；陪练要记得住，而且不越界。

第一个动作是划边界，不是写代码。有两件事我一开始就决定不做。一是**美术与完整游戏内容**：规模控制在 14 只宠物、7 种属性、5 个关卡，这个量已经足够产生真实的取舍——是先抢着收尾，还是把能量留到下一回合；是换一只上来承伤，还是原地吃药。二是**联网 PVP**：本地对战是同机分屏，双方各自锁定行动，然后一起结算；服务端只接收浏览器传来的状态快照，不掌握权威对局。这条差距是架构问题，不是工程量问题。

三种角色我全部做完了，题目只要求闭环其中一种。它们的难点并不重叠：军师的难处在分工，哪些判断交给引擎、哪些话交给模型；老师的难处在时间口径，复盘只能用回合开始前看得见的东西；陪练的难处在克制，既不越界去当军师，也不评价玩家。只做一种，另外两类的毛病不会暴露出来。

三种角色共用一条执行链

军师、陪练、老师是三种角色，跑的是同一条链：玩家提问、或代码判定该开口时，请求先过门控，再由有界的工具循环去查证，查到的回执必须过校验，最后才允许输出——或者允许沉默。三种角色的差别只在提示词、证据包与措辞约束上，链本身只有一条，七步各自的出处见图 2。

链上七步各有出处：触发分两路——玩家提问走聊天面板，自动触发由 `app.js` 的事件钩子与 `coach/experience.js` 的触发层判定（每局上限、理由去重、冷却都在这一层）；门控在 `policy.js`；循环、校验与输出上限都在 `runtime.js`；工具与参数契约在 `toolbox.js`；被查的证据来自 `engine.js` 枚举出的合法行动与本地知识卡；记忆写在 `memory.js`。工具路由头、干预时机、检索对照这三组实验产物由脚本跑到结果，不在上面这条链上。

记忆与反思这一环要把两件事分开。**规则与证据驱动的 Reflection 已经实现**：同类独立行动满 3 条时，`rememberDecision` 写下一条掌握判断，带着依据；它真的改变行

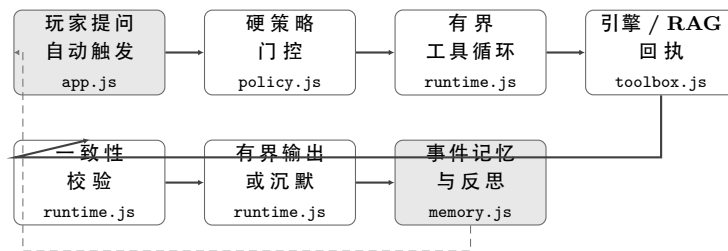


图 1 三种角色共用的执行链，以及链上七步各自的出处

为——adaptiveGate 据此降频，observeStruggle 与 markUnlearned 据此重教。LLM 自动总结式 Reflection 没做，Generative Agents 式的自由合成不在这一版里。

2 交付了什么

军师、陪练、老师是三种角色，跑的是同一条链：玩家提问、或代码判定该开口时，请求先过门控，再由有界的工具循环去查证，查到的回执必须过校验，最后才允许输出——或者允许沉默。三种角色的差别只在提示词、证据包与措辞约束上，链本身只有一条，七步各自的出处见图 2。

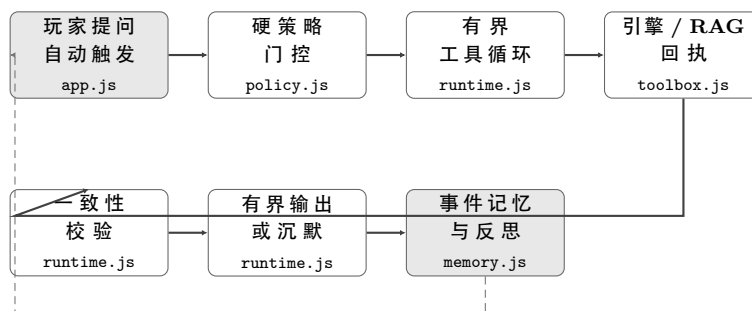


图 2 三种角色共用的执行链，以及链上七步各自的出处

链上七步各有出处：触发分两路——玩家提问走聊天面板，自动触发由 app.js 的事件钩子与 coach/experience.js 的触发层判定（每局上限、理由去重、冷却都在这一层）；门控在 policy.js；循环、校验与输出上限都在 runtime.js；工具与参数契约在 toolbox.js；被查的证据来自 engine.js 枚举出的合法行动与本地知识卡；记忆写在 memory.js。工具路由头、干预时机、检索对照这三组实验产物由脚本跑到结果，不在上面这条链上。

记忆与反思这一环要把两件事分开。规则与证据驱动的 Reflection 已经实现：同类独立行动满 3 条时，rememberDecision 写下一条掌握判断，带着依据；它真的改变行为——adaptiveGate 据此降频，observeStruggle 与 markUnlearned 据此重教。LLM 自动总结式 Reflection 没做，Generative Agents 式的自由合成不在这一版里。

交付的是一个能直接玩起来的东西，教练叫小芽。游戏本体很简单：训练与本地对战两种模式，营地负责养成与配招，出征时选关卡、按顺序挑三只伙伴，局内逐回合出招。小芽的三个角色共用同一份对局局面、同一份记忆，区别在位置和分量：军师和老师在顶部那条提示里，各说一句短话，点开才展开依据；陪练在左下角，自己找时机开口。

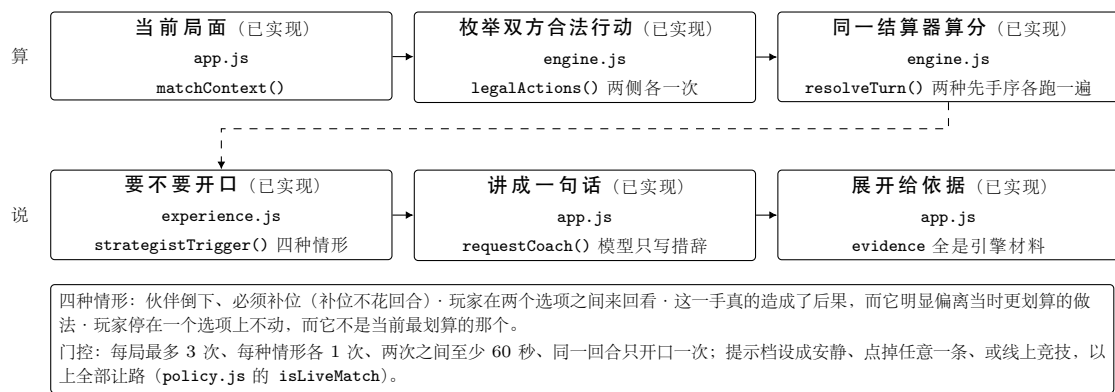


图 3 军师的机制：先把这一回合算完，再决定说没说、怎么说

2.1 军师

军师做的是把这一回合的取舍摆出来，不替玩家出招。玩家看到的第一行永远是一句短话，比如「这一回合优先考虑『火花』，可比较的备选是『余烬追猎』」；往下点开才是依据。话是模型写的，依据不是：引擎把双方这一回合所有合法行动列出来，每一种组合都放进同一个结算器里真跑一遍，算出一个分数，分数高的排前面，模型只负责把结果讲成玩家听得懂的话。这条链见图 3。

这个分数只用来排序，不是胜率，它只管这一回合。双方速度相同时，谁先出手会改变这一回合的结果，所以两种先后顺序都各跑一遍，两个结果一起算进去。

军师也会主动开口，但只在四种情形下：伙伴倒下、必须补位（补位不花回合）；玩家在两个选项之间来回看；玩家这一手真的造成了后果，而它明显偏离当时更划算的做法（手上有能一击收尾的招，却没打）；玩家停在一个选项上不动，而它不是当前最划算的那个。每局最多开口 3 次，每种情形各 1 次，两次之间至少隔一分钟。把提示档设成安静（只回应提问），或者在对局里点掉任意一条，上面这些全部让路。

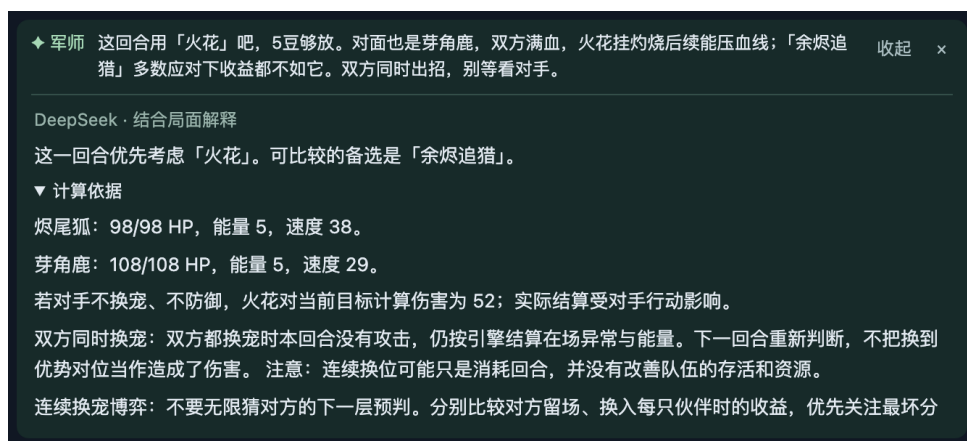


图 4 军师提示条与它展开的计算依据

图 4 是其中一条提示展开之后的样子。下面那块「计算依据」全是引擎给的材料：双方当前的血量、能量和速度，这一招打在当前目标身上算多少伤害，以及这条判断引用了哪两张知识卡——每张卡都带一句它自己的反例。依据里不会出现对手这一回合要出的那一手：两边锁定行动之前，军师看不到，也不允许猜。

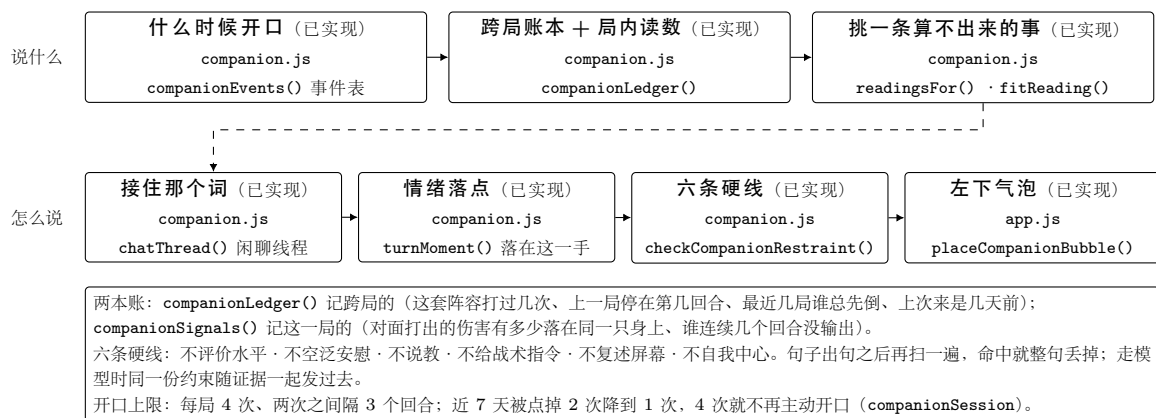


图 5 陪练的机制：从两本账里挑一件玩家自己算不出来的事，再决定怎么说

2.2 陪练

陪练是三个角色里唯一一直在场的那一个。它不等人召唤，也不是点开才出现的客服：对局进行中它会自己说话，位置在战斗页左下角，独立于聊天面板。每一条都能点掉，点掉之后这一局它就不再开口。主动开口有上限：默认每局 4 次，最近七天被关掉 2 次降到 1 次，关掉 4 次之后这一局就不再主动说话。这条链见图 5。

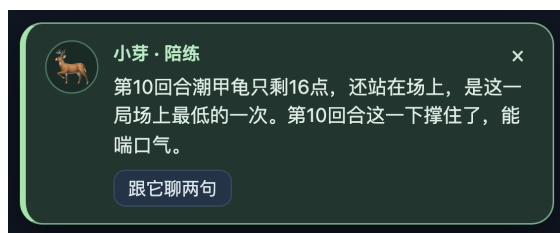


图 6 陪练气泡：不打开聊天也能看见，可以点掉

它说出口的每一句，都要落在了一件玩家自己算不出来的事上。一类是跨局记录：这套阵容打过几次、上一局停在第几回合、最近几局最先倒下的总是哪一只、上次来是几天前。一类是跨回合的统计：对面这一局打出的伤害有多少落在同一只身上、某一只连续几个回合没打出输出。图 6 里那句是第二类——「是这一局场上最低的一次」这个判断，得把整局每一回合的血量都比一遍才说得出来，屏幕上的血条只显示此刻的数值。

情绪是它和另外两个角色最不一样的地方，但情绪必须有落点。它只会用五种态度：可惜、漂亮、悬、憋屈、松口气；每一种都要绑在刚结算的那一手上，说清是第几回合、哪一只、哪一记技能。比如一记收尾：「第 6 回合那记余烬追猎，把还剩 55 点血的炽鬃狮一记收掉了。整局你都没打出过这么重的一下。第 6 回合这一下，漂亮。」反过来，讲它自己的感受是不允许的：「我看得有点急」这类句子会被拦下来，它把玩家变成了来看它着急的旁观者。

它记得的事全部来自本机记录，缺一条就不说，不编。连败几局、最近赢没赢、上一局打到第几回合、还剩几瓶回复药、本命是哪只、上次玩是几天前，加上玩家自己声明过的偏好和目标。这些字段决定它用哪一档语气：平常就事论事，玩家主动问就多说一层，连着输了两三局，别的话就收起来，只陪着坐一会儿，收尾那句是「到这儿也行，先歇会儿」。档位本身也写进模型看到的证据里，它说出的每一句都能回到一条本机记录上。

闲聊是它的另一个出口。玩家主动搭话时，它先接住玩家自己用过的那个词，再往

下说：一句「今天有点累」，回的是「今天累了啊——那就先歇着，不用急着做什么」。这一轮里不报战绩、不提回合数，也不问对局。这是「每句话都要说一件真事」唯一一处例外——人在说心情的时候，要的不是数据。问候和道谢也一样，先应一声：「你好」回的是「你好，我是小芽。你打过的那 3 局我都留着底。」

它的边界写得很死：不给战术指令（那是军师的事），不评价玩家打得怎么样，不说教，不空泛安慰。这几条是硬线：句子出句之后还要再扫一遍，命中就整句丢掉，走模型时同一份约束随证据一起发过去。它和军师可以同时说话：军师在顶部条上给战术，它在左下角陪着，两处位置不同，说的也是两回事。

2.3 老师

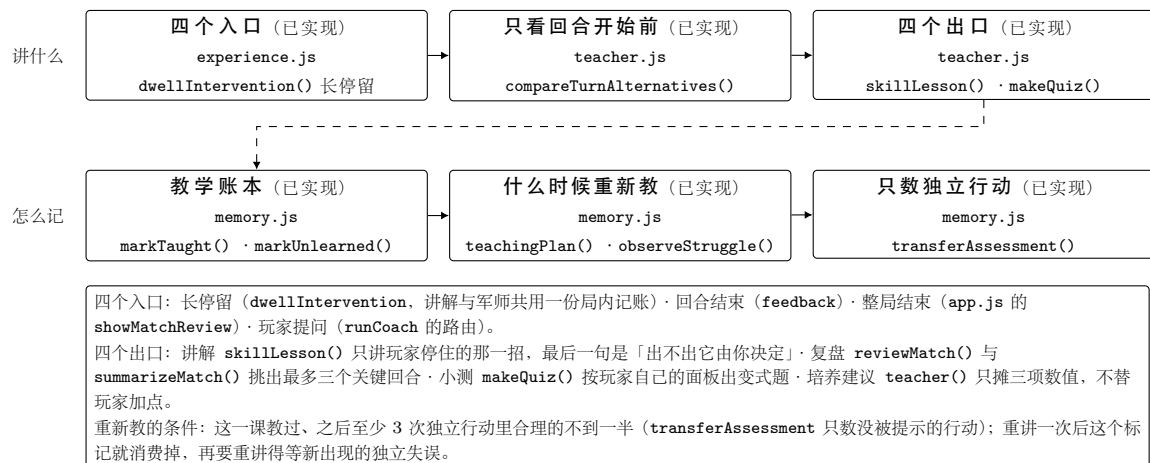


图 7 老师的机制：只用回合开始时的公开局面，讲完记进教学账本

老师的出口有四个：局内讲清玩家停在哪个技能上，复盘一个回合或一整局，按玩家自己的面板出小测，以及在营地里给培养建议。这条链见图 7。

局内它只讲玩家停住不动的那一个技能：什么属性、要花几豆、打在当前目标身上算多少伤害、打完还剩几豆，以及它在什么条件下比手里别的选项更合适。它只解释这一招，不催出招，最后一句写着「出不出它由你决定」。如果玩家停的那一招恰好是当前最划算的，讲解的活归老师，军师不开口——同一件事不安排两个人说。

整局结束的复盘是它最重的一块：上面一行是这局的结论，展开区里是按减员和生命变化挑出来的最多三个关键回合（图 8）。每个回合写清当时双方还剩多少血、这一手实际打出了什么，以及当时还能选哪些做法、它们比实际这一手好多少。整局统计只讲发生过的行动：没换过宠、没用过道具就不提。

复盘有一条硬口径：一律只看回合开始前的公开局面，不用对手实际打出的那一招。否则任何一次失败都能被解释成「你当时选错了」。把对手那一步换掉，复盘的结论不变，这一条有测试守着。

小测按玩家自己的面板出变式题，不背题库。面板速度 38，培养一次敏捷是 41，对手速度 41，答案就是「无法确定」——同速时谁先出手由规则里的随机过程决定，不能保证先手；选项里专门留着「不确定」这一项。培养建议是它唯一涉及成长的入口：把三项数值加一点之后的对照摊开——生命 98 到 110、攻击 27 到 31、速度 38 到 41（图 9）——并说明这一次加点会不会改变与当前关卡首发的先后关系。加点这件事由玩

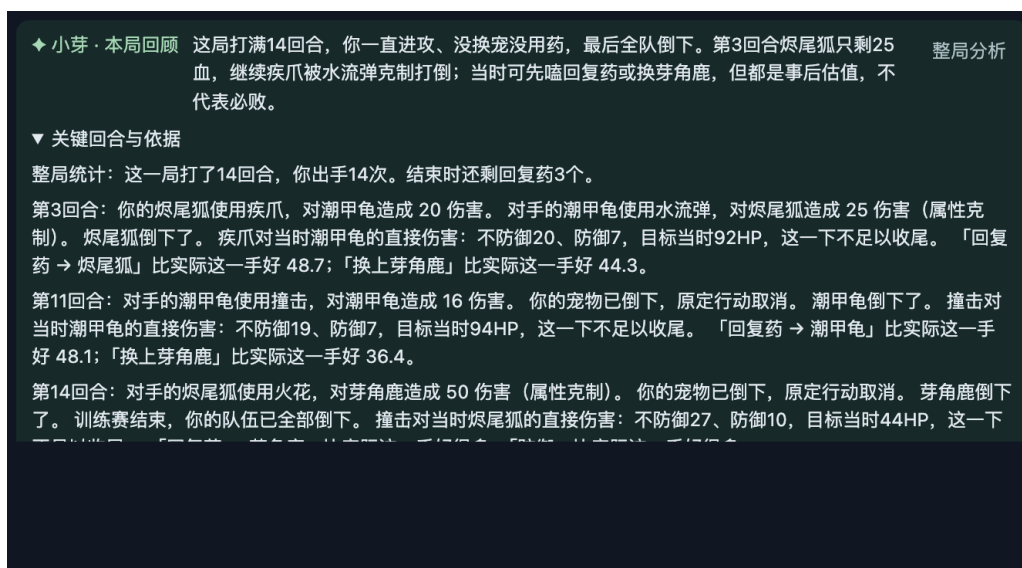


图 8 老师的整局复盘与挑出来的关键回合

家自己按，老师只给数。

项目	现在	加1点后
生命	98	110
攻击	27	31
速度	38	41

图 9 培养建议：三项数值加一点之后的对照

真正花心思的是教学账本，它记三件事：讲过哪一课、哪一课被标回未掌握、这一课被提示过和没被提示过的行动各有几个。讲过又没有新的反证，就不重复讲。只有当军师在局内独立看到三次以上同类失误——至少 3 次独立行动、其中合理率不到一半——这一课才会被标回未掌握，它也说得出为什么：「这一课教过，但之后 4 次独立行动里有 3 次不合理」。重新讲过之后这个标记就消费掉，再要它重讲，得等新出现的独立失误。

教学账本只数没被提示过的行动，理由来自一篇讲 assistance dilemma 的论文。提示本身会把学生推向高效步骤，他们早期的预测器没把「学生用过提示」算进去，于是用过的学生反而被判成「不需要帮助」；修正的办法就是把两者分开。小芽这里是同一件事：判断玩家有没有掌握一个知识点，只看他在没有被提示的时候做出的选择。这套账本是自建的计数与规则，不是心理测量模型：它只回答这一课教过没有、这一次要不要再教，不估计掌握概率。

3 协作中的难点与解决

分工是清楚的：我负责顶层设计、真实使用与验收，子 agent 负责具体实现和代码层面的测试。架构、三种角色的边界、每条约束的判据由我定，单元测试交给子 agent 写，可以并行推进。代价是「实现完了」和「功能生效了」是两回事——子 agent 写得出堆单元测试，这些测试能验证一个函数内部对不对，验证不了这个函数有没有被接上，也验证不了它量的是不是该量的东西。这一章里的问题都是这么漏出来、又被我一个一个测出来的。

3.1 代码是对的，功能没有生效

同一个形态出现过三次：代码看着对、`npm test` 全绿、功能根本没有生效。

- 子 agent 新增的模块 `rules.js` 没有进静态资源白名单。磁盘上的白名单、测试、模块图全是对的，但我打开页面看到的还是白屏：运行中的那个进程拿的是旧白名单，`/rules.js` 返回 404，`app.js` 的 `import` 图整体失败，页面一行 JS 都不执行。子 agent 的测试只能验证磁盘上的文件，验证不了我正在访问的是哪个进程。
- 陪练的主动气泡模板写好了，调用它的 `notify()` 却没有任何地方调用过，所以气泡从不出现。函数本身的每条测试都通过，因为没有一条测试问「谁调用它」。
- 军师里留了一个未定义变量，控制台每秒抛一次 `ReferenceError`——子 agent 改动时删掉了它的定义，留下了引用。

这三条都不是读代码读出来的，只有我把页面打开、真打一局才看得见。

3.2 视觉一致：只改我点名的那一处

界面上的不统一，每次都得我自己开页面对一遍，逐条说、逐条验。招式卡高矮不一，原因是一张卡里描述长短不同，整排跟着参差；对局页左右两侧的格式也要一致，两侧标题长度不同，会把各自的筛选条推到不同的起点。我第一次说这件事时把方向指错了，子 agent 照着改了一遍间距，问题还在；真正管用的是给标题固定宽度。筛选按钮我要求对称、撑满本栏，与下面的伙伴网格同宽。还有「我方」两个字，写在分屏标题里既没用又不对齐；`x` 标记也是这样，改成绝对定位之后，已经选进队伍的卡不再因为多出一行而变高。

这些改动的代码量都很小，难的是我每次只能指出我看到的那一处，子 agent 就只改那一处，下一次我再看见另一处。

3.3 重复信息：同一句话出现在几个地方

同一句话出现在几个地方，只有真的用起来才会撞上。出征页顶部的徽标写着「训练·PVE」，右栏又写了一遍同样的模式；回合结算后，横幅把刚在动画里念过、也已经写进战斗记录的那一帧复述一遍；最严重的一次，一段老师的讲解同时出现在顶部条、左下浮层和右下气泡三处，文字一模一样。还有一类更小的：同一个动作在界面上有三个名字，标签写「认输」，按钮写「确认撤退」，战斗记录里又写成「撤退」。这类问题要开页面逐处看是看不完的，得把玩家可见的文案集中到一张表上逐条对，「同一件事」

才浮出来。

3.4 验收要在真浏览器里真打一局

我的做法是加两层检查，都不进 `npm test`——需要浏览器的检查不该当成必过门。

第一层是接线检查 (`wiring.test.js`)，由子 agent 写：`app.js` 里每个 `$(‘id’)` 都必须在 `index.html` 或 `app.js` 自建的元素里存在，死函数清单必须恰好是两个已知遗留，面试题要求的每项能力都必须找得到入口；它第一次运行就抓出一处指向已删除元素的残留引用。第二层是冒烟测试，同样由子 agent 写：无头浏览器从营地选人、逐回合出招、打到结算，断言页面渲染出伙伴卡、对局有结果、控制台零报错。

3.5 验收的另一半是核对口径

检查只能验证断言本身，验证不了断言在数什么；有几处缺陷的代码完全正常，错的是口径。玩家补位的那一回合本来就不该出招，却被计成一次「引擎兜底」，方向恰好让对手 agent 显得比实际更差。犹豫判定用的是「距第一次悬停的墙钟时间」：我在火花上停了一下，然后移去对方面板操作了 13 秒，回来碰了「防御」，界面就弹出「你在火花和防御之间停留了 13 秒」——那 13 秒我根本没在看这两个选项。信号要量的是花在这上面的时间，不是经过了多少时间。这两个例子不会让任何测试失败，只能逐条核对分子分母才能发现。

还有一类要读文字才看得出来。`README` 里写着「模型不参与伤害计算，也不决定对手出招」，而代码里对手的出招正是模型从引擎给出的合法行动表里选的——全仓库没有一条测试会去读 `README`。这类错处只能对着产物一条条重算；我专门做了一遍全量核对，清出来的错处里有好几条属于这一类。

3.6 检查本身也要被检查

比漏改更麻烦的是：子 agent 用来兜底的检查本身就不起作用。

冒烟测试里的伙伴卡数量写死过，宠物增加之后那个数字不再对得上；语义检索读的语料是手工维护的第三份副本，两张卡改了它不知道，模型引用到的仍是旧文案；工具参数用合法行动列表的下标，而索引会随能量、道具和倒下整体位移，同一个下标在不同回合指向不同行动，回执里也没法核对。

玩家文案里不许出现「启发式」「枚举」「分支」这类内部词。这一条我提了要求，子 agent 照此写出了禁止词表检查；但它的检查有盲区，是我在核对时发现的。按我的要求它写完先做负向验证，即便过了这一关，检查取局内提示时仍然没有传事件参数，那条理由的正文根本不在覆盖范围内；第一次修补也没修对，子 agent 手工造一个事件参数传进去，负向验证立刻戳穿。最后按我的意见换成静态扫描：逐行取 `coach/` 下的中文字面量对词表，不依赖运行时路径，随即又扫出四处真泄漏。

这几条形态不同、毛病一样：检查写死了预期值，或者与被测对象不同源，或者只覆盖跑得到的那条路。所以每写完一条检查，我会先做一次负向验证，把错的东西放进去，看它会不会失败；再让期望值从被测对象自己推导出来，不从手写的常量来。检查写完之后，检查本身还要过一遍同样的关卡。

3.7 说了很多遍「已完成」，其实没生效

子 agent 的每一次交付都配着一句「已完成」，而我每次都得自己再验一遍。最典型的一次是角色路由：我指名要陪练，回答却从老师那条路出来——`coach/client.js` 在发送前会追加一段请求说明，里面含有「回合」，而路由判断先匹配这个词，于是角色被改判。单元测试测的是函数，测不出「是不是它被调用了」。现在每个能力都有一条能执行的入口检查（`wiring.test.js` 逐个断言），改完路由我必须自己在浏览器里跑一遍。光重申没有用，只能把它变成机械检查。

4 交付结果

结果就是小芽本身：一个住在游戏里的教练。一局游戏从头到尾，玩家不打开聊天面板也会被它接住；三种角色各自跑完了自己的闭环——军师在局面里给可解释的取舍，老师在一局结束后复盘并把课上下去，陪练在左下角陪着，记得住、有情绪、不越界。

军师：每一步的取舍都算过，也说得出依据。轮到玩家出招时，引擎已经把双方所有合法行动枚举过一遍，各是什么后果都算过；模型只把这堆事实讲成一句话。点开「看看原因」，展开的就是它引用的血量、能量和知识卡。它也知道什么时候该闭嘴：安静档下只在被问时回答；主动开口只留给五种局面——伙伴倒下、来回犹豫、明显是错的一手等；每局有次数上限、同类不重复、带冷却，点掉就本局静音。

老师：复盘、讲解、小测，还带一本教学账本。一局打完，它先给一句话结论，再从减员和生命变化里挑三个回合逐个讲。讲评只用回合开始时的公开状态，不拿对手打出的那一招当理由——否则每次失败都能说成「你当时选错了」。局中它盯着玩家正在看的那个技能讲，不催出招。小测用真实面板出变式题，选项里带「不确定」。要不要再讲一遍只看有没有新的反证：讲过又没有就不重复，只有军师独立看到三次同类失误才重新开课，并说得出是哪三次。

陪练：记得住、有情绪、不越界。它在战斗页左下角，一个独立气泡，不打开聊天也看得见；点开就是聊天，玩家只说「你好」它也接得住。情绪必须落在真实事件上：能说的只有可惜、漂亮、悬、憋屈、松口气五个词，每一句都绑在一条本机读数上；它不说自己的感受，「我看得有点急」这种句子会被拦掉。它记得住连败、药还剩几瓶、本命是哪只、上次玩是几天前，还有玩家声明过的偏好和目标，据此决定语气档位。边界是写死的：不评价玩家水平、不给战术指令、不说教；每局主动搭话有上限，最近七天被点掉的次数还会把上限压低。

检索：够用，但不够准。它靠本机的知识卡库找依据，不与模型绑在一起。找得到的，军师和老师的解释就引得出卡片；找不到的，它本该说不知道，实际却常常硬答——我试过一批库里没有答案的问题，一半以上它照样给了一个答案。同一个问题的不同问法，有时也排不到前面。加大词表能挪动排序，但赢不了多少，也不稳定。更完善的语义检索在仓库里有一份独立实现，产品当前没接。

模型不在，功能也不会消失。规则建议、复盘、培养建议和小测都由本机引擎给，不依赖模型。模型没接上、超时、输出过长，或者和工具回执对不上，屏幕上显示的都是本机已核验的结论，并说明为什么退回——不会把一句对不上的话留在那里。

它可靠到什么程度。每条能力都有对应的自动测试，`npm test` 一条命令跑完，盯的是行为不是文案：该不该开口、数字和引用的卡片对不对得上、越界的话有没有被拦下、提示有没有立刻失效。真实模型跑过的回执都留在 `reports/` 下。不过测试证明的是「在我写死的局面上代码没有出错」，不是「玩家真的学会了」——那要真人被试，看撤掉提示之后还能不能自己做对；检索评测的题目和标注也是我自己写的，绝对数字偏乐观。语音没做，联网对战是同机分屏，仓库里的演示是无头浏览器抓的截图，不是真人录屏。

5 运行与使用

代码在 <https://github.com/SeRendizc/pet-game-coach>，Node.js 20+，没有第三方依赖。

- `npm start`：启动，浏览器打开 <http://127.0.0.1:8765/> 就能玩
- `npm test`：341 项自动测试，几秒跑完；`npm run test:smoke`：无头浏览器真的打一局

不接模型也能完整游玩，规则建议、复盘和小测都由本机引擎给。想接模型，就打开 <http://127.0.0.1:8765/connect.html> 填上 DeepSeek API Key，密钥只留在后端进程内存里；也可以先跑一次 `./scripts/start.sh --save-key` 存进钥匙串。

想看某个能力：营地是培养和配招，出征先选关卡再挑三只伙伴；对局里，军师的提示在战斗页底部，点「看看原因」能看到依据，陪练在左下角的气泡和聊天里；一局打完看复盘和关键回合，培养页有加点对照和小测。